Ридж-регрессия

Часто при наличии в данных сильных зависимостей между разными параметрами обычный алгоритм теряет устойчивость (так как в вычислениях появляются очень малые и очень большие числа). Можно бороться с этим, дополнительно ограничив максимальные размеры параметров в минимизируемой функции:

$$E = (XC - Y)^T(XC - Y) + \lambda C^T C$$

λ — фиксированный коэффициент. При нулевом значении получится то же, что и раньше; чем больше λ, тем сильнее будут уменьшаться параметры.

Решение будет иметь вид:

$$C = (X^T X + \lambda I)^{-1} X^T Y,$$

где I — единичная матрица.

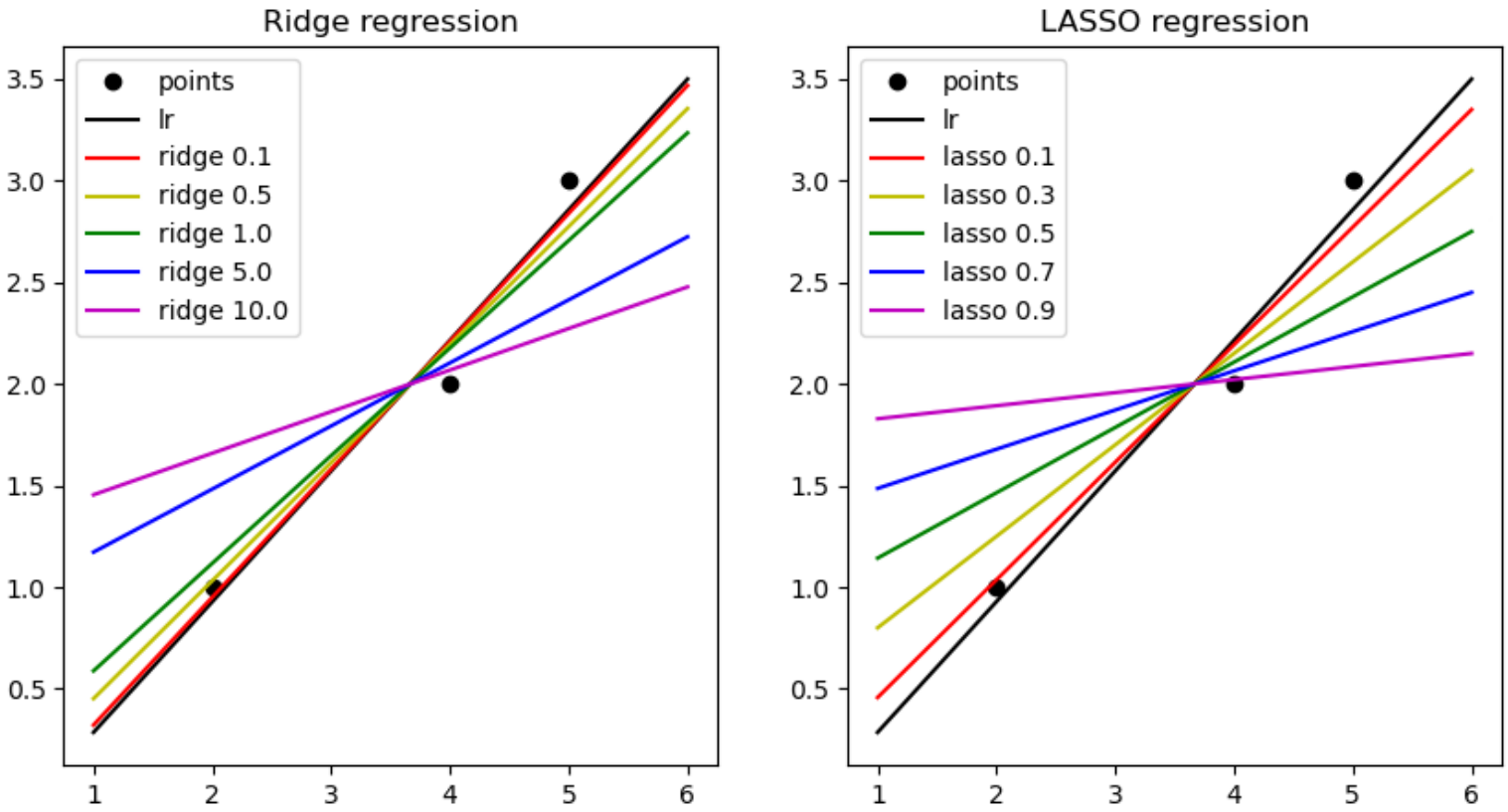
Лассо-регрессия

LASSO (Least Absolute Shrinkage and Selection Operator) — также борется с бесконтрольным ростом параметров, но иным способом. К стандартной функции минимизации добавляется ограничение суммарной величины параметров:

$$\sum_{i=1}^{K+1} |c_i| \leq \lambda$$

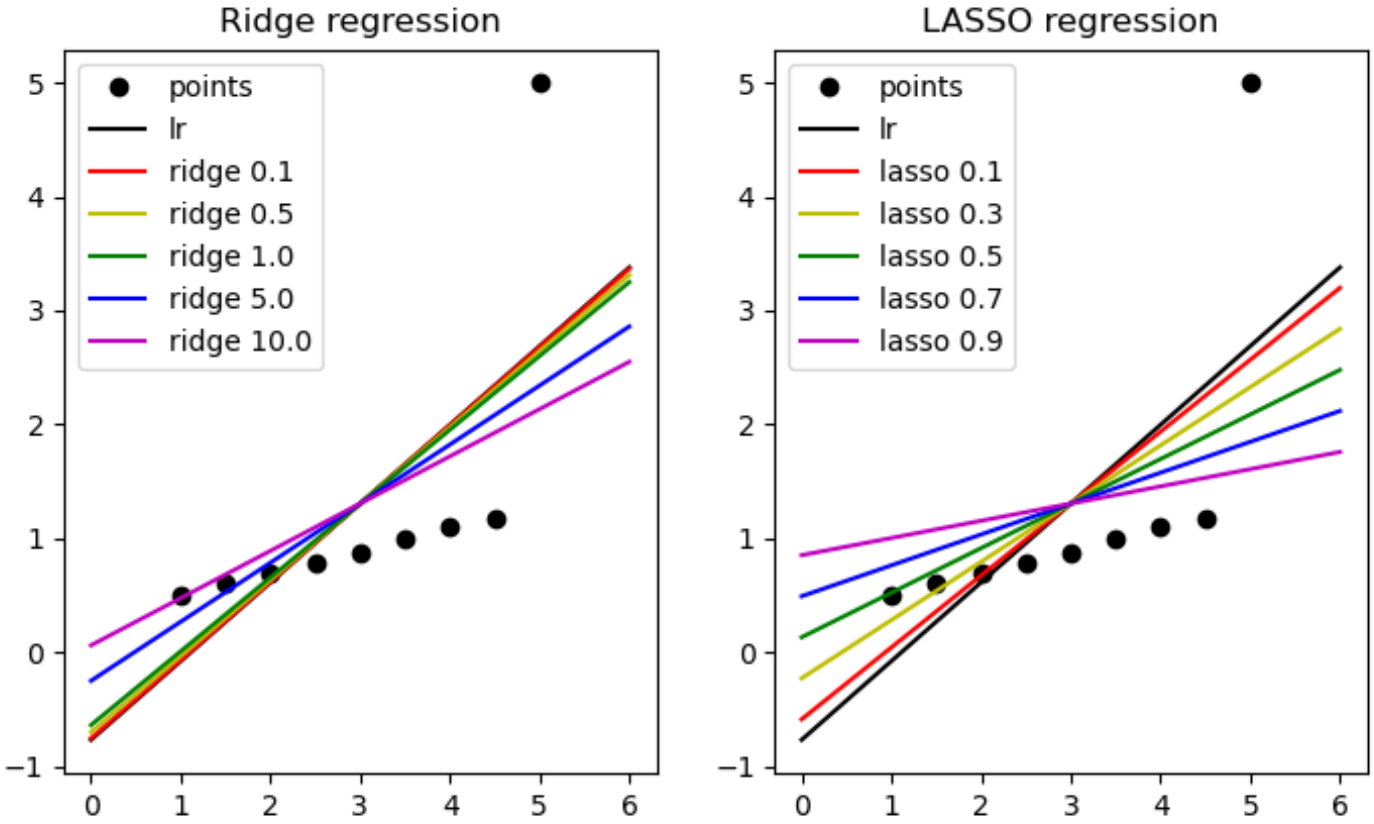
где λ — параметр.

Пример регуляризации (1)



Видно, что коэффициенты регрессии уменьшаются при росте параметров

Пример регуляризации (2)



Иногда удаётся при этом снизить влияние выбросов

Итоги по методу наименьших квадратов

Достоинства: можно получать точные и строгие результаты, подходящие под критерий максимального правдоподобия.

Недостатки: неустойчивость к выбросам — резко отличающимся значениям.

Решения:

- отбрасывать выбросы заранее;
- использовать критерий, отличающийся от метода наименьших квадратов — например, минимизировать не сумму квадратов ошибок, а сумму их модулей.

Методы анализа данных, которые устойчивы к выбросам, называются **робастными**.

Минимизация суммы модулей ошибки

Что, если минимизировать сумму модулей ошибок, а не квадратов?

$$E = \sum_{i=1}^{K} |g(x_i, w) - f(x_i)|$$

Вопрос: если хочется применить метод максимального правдоподобия, какую форму должно иметь распределение?

Минимизация суммы модулей ошибки

Что, если минимизировать сумму модулей ошибок, а не квадратов?

$$E = \sum_{i=1}^{K} |g(x_i, w) - f(x_i)|$$

Вопрос: если хочется применить метод максимального правдоподобия, какую форму должно иметь распределение?

Ответ: по аналогии с гауссианом, имеющим множитель e^{ε^2}, нужно использовать распределение с множителем $e^{-|\varepsilon|}$. Если действовать с ним точно так же, как выше с нормальным распределением, получим максимально правдоподобное решение в виде минимума суммы модулей ошибок (а не их квадратов). Такое распределение называется распределением Лапласа.

Прочие виды регрессии

RANSAC
Сплайны
Ядерное сглаживание
Гауссовский процесс для регрессии
Реализация моделей на scikit-learn

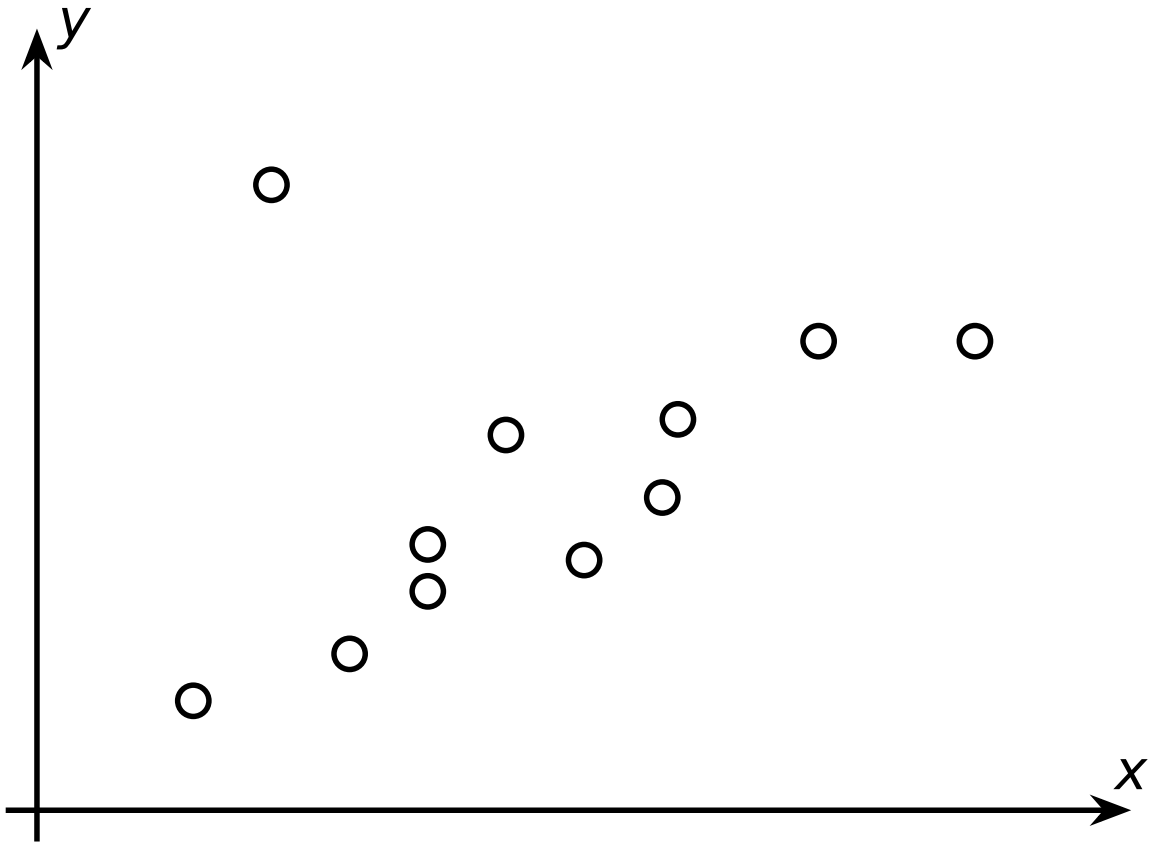
RANSAC

RANdom SAmple Consensus: более примитивный подход, но применим к более широкому классу задач, в которых требуется 1) найти подходящие для заданной модели прецеденты и 2) отделить их от выбросов.

Базовая идея на примере распознавания прямой линии в облаке точек на плоскости:

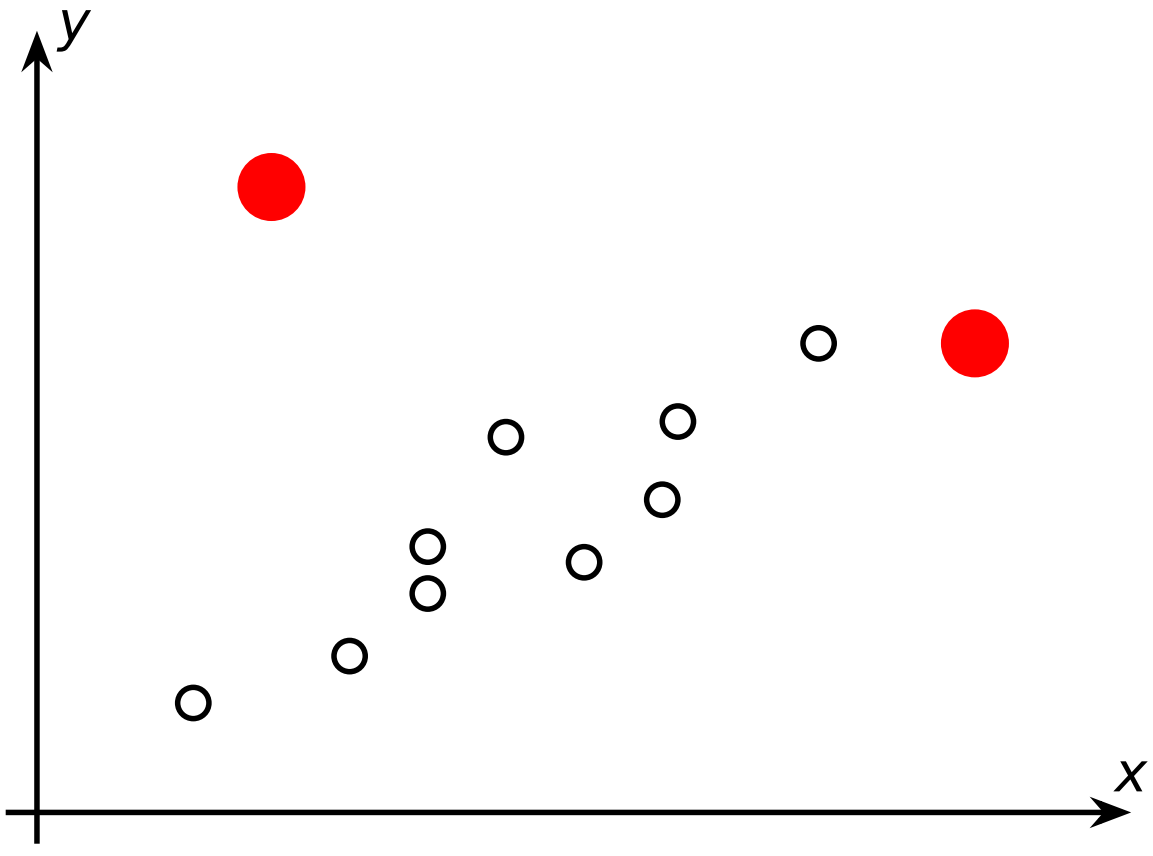
1. возьмём любую пару точек и посчитаем, что они лежат на искомой прямой;
2. для всех остальных точек рассчитаем расстояния до полученной прямой (или, возможно, уклонения); если расстояние достаточно мало, точка лежит на прямой, иначе она является выбросом;
3. если достаточно много точек лежит на прямой, задача решена; можно ещё дополнительно уточнить уравнение полученной прямой, если это требуется;
4. если прямая содержит мало точек, вернуться к шагу 1; повторять его либо до успеха, либо неколько раз.

Пример работы RANSAC



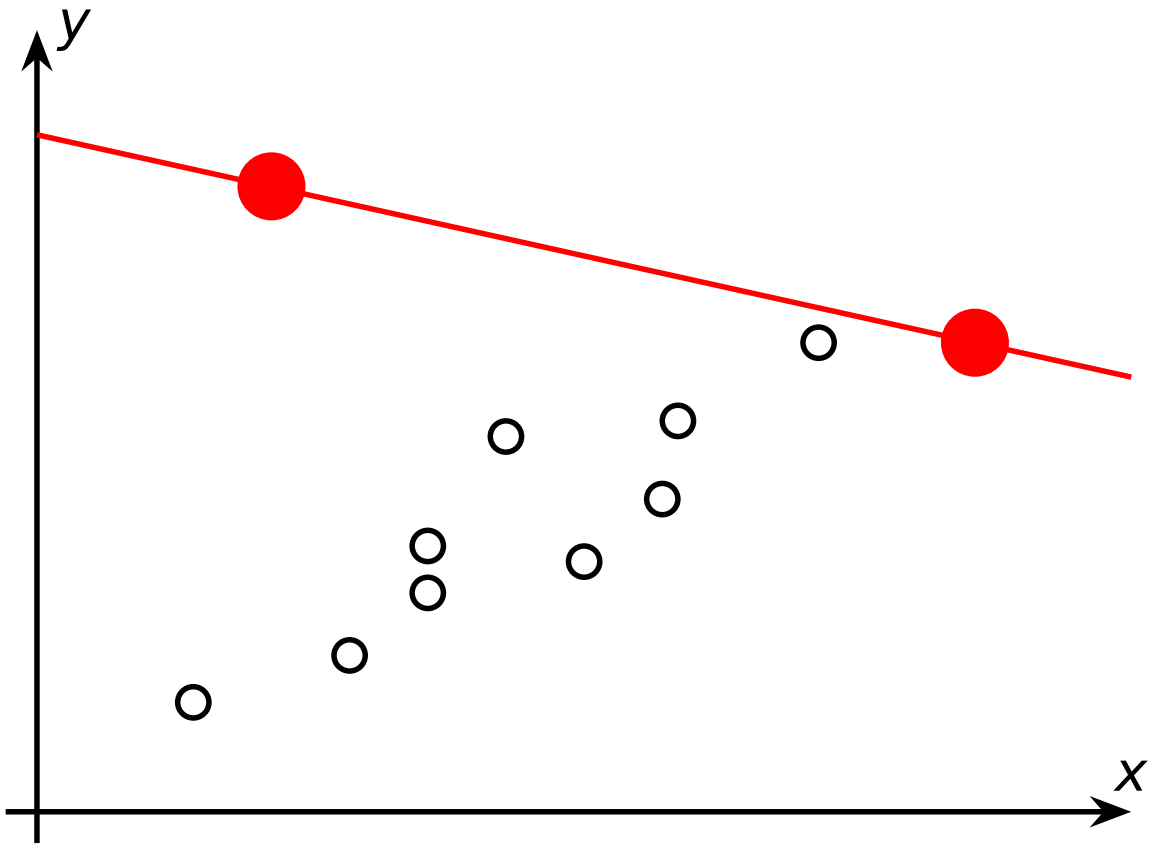
Пусть требуется провести прямую, близко расположенную к показанному множеству точек

Пример работы RANSAC



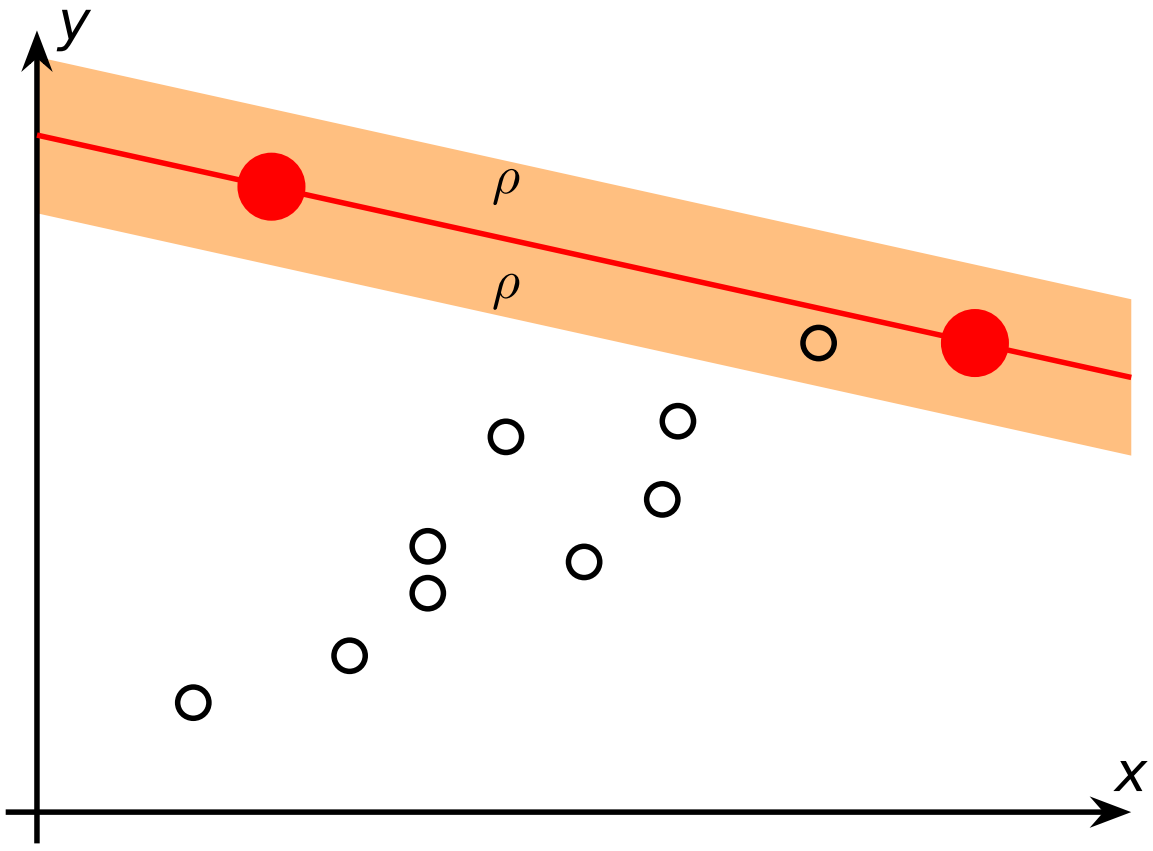
Для того, чтобы однозначно задать прямую, нужно ровно две точки. Выберем их случайным образом из имеющихся

Пример работы RANSAC



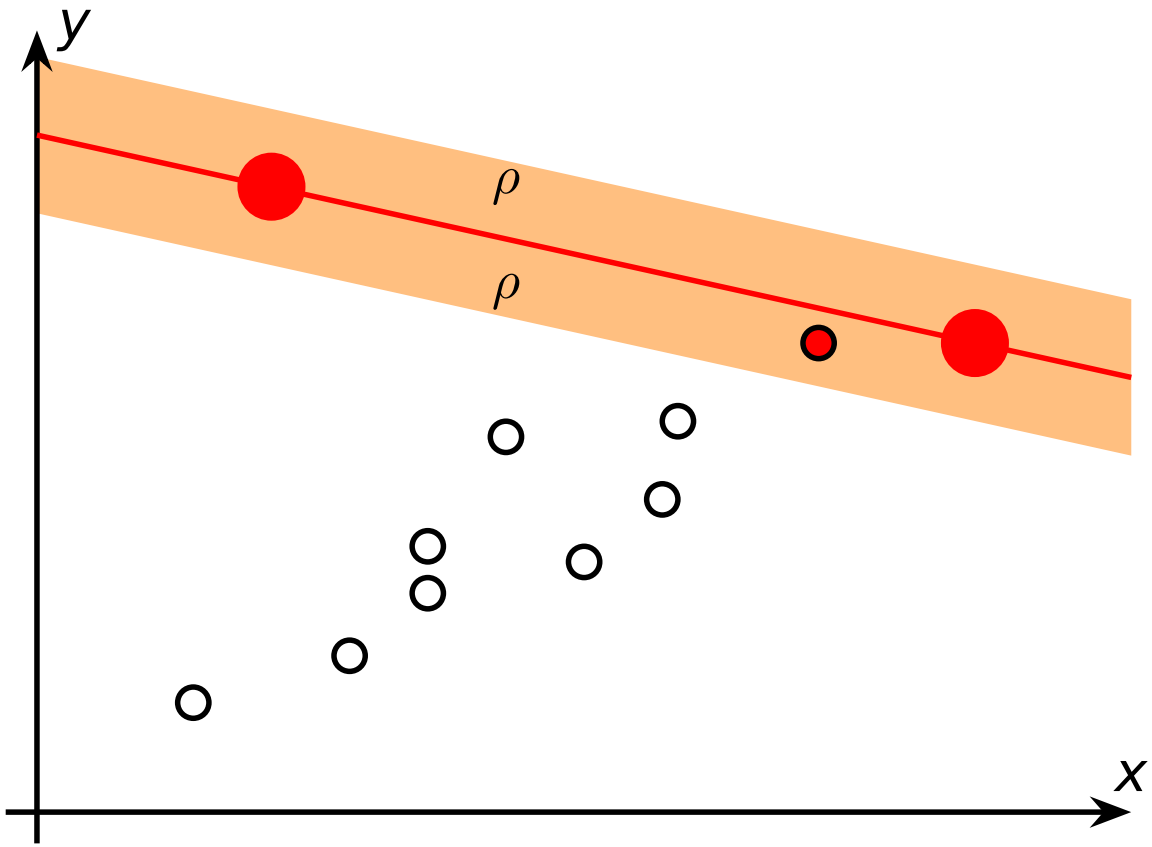
Проведём через них прямую и проверим, много ли точек лежат достаточно близко к ней

Пример работы RANSAC



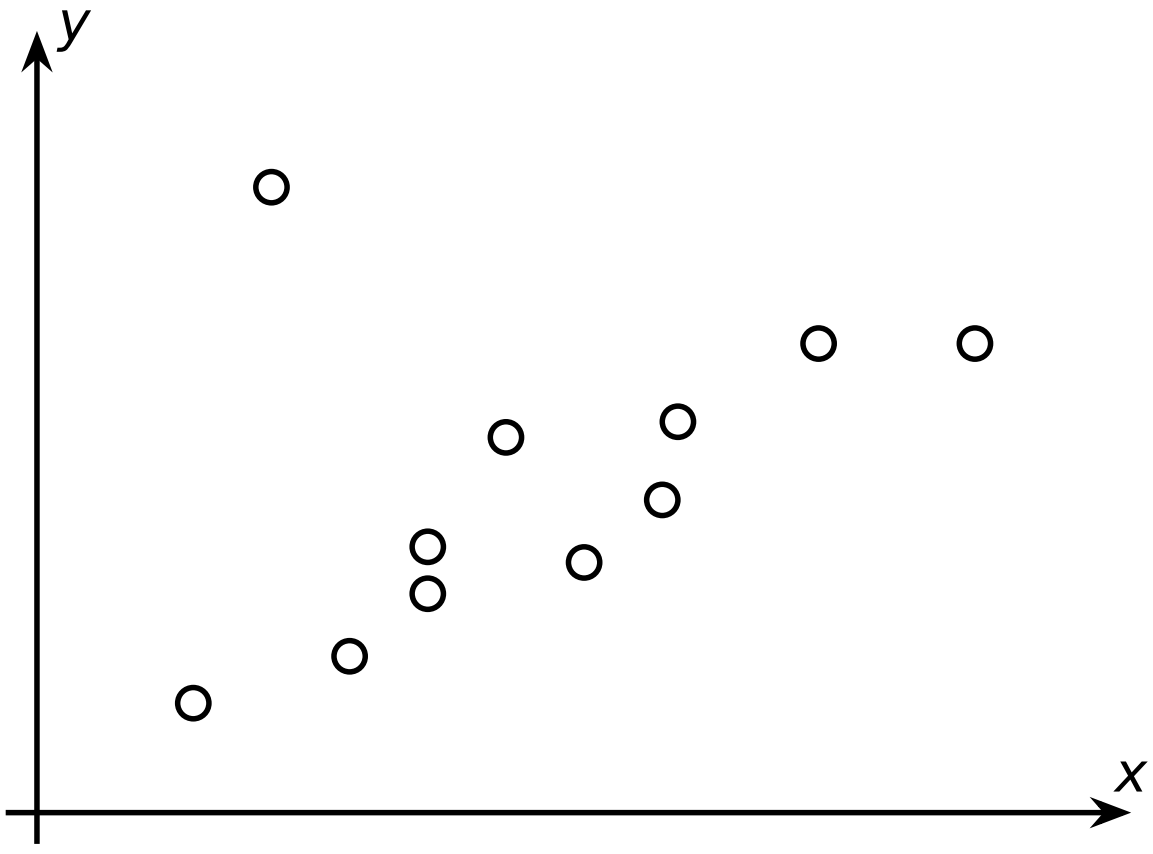
Более формально: заранее зададимся предельным уклонением точки от прямой ρ: если точка находится от прямой на расстоянии не больше ρ, можно считать, что точка лежит на прямой, иначе — что точка лежит вне прямой

Пример работы RANSAC



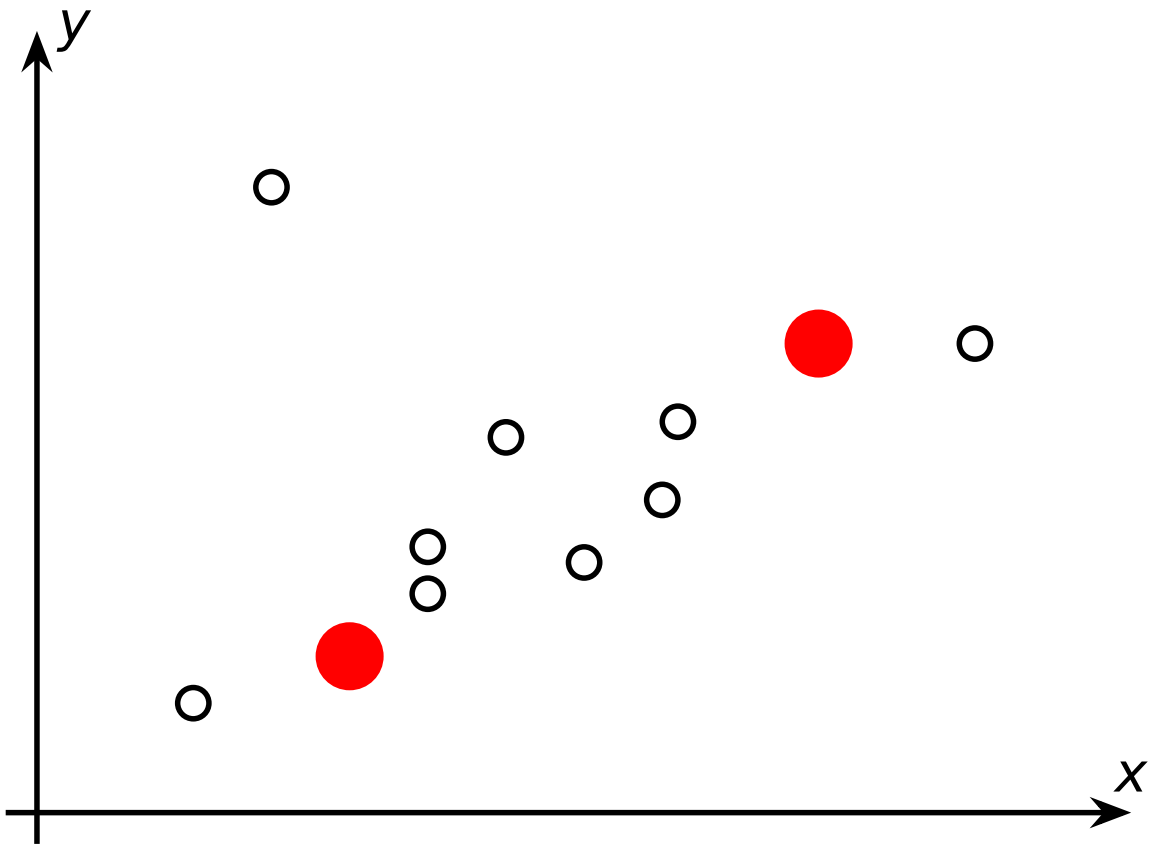
Видно, что в полученной полосе шириной 2ρ лежит очень мало точек. Чем их больше, тем прямая больше похожа на нужную; полученная нами прямая весьма плоха. Обозначим как ψ минимальную долю точек, которая должна лежать на прямой, чтобы эту прямую можно было считать решением.

Пример работы RANSAC



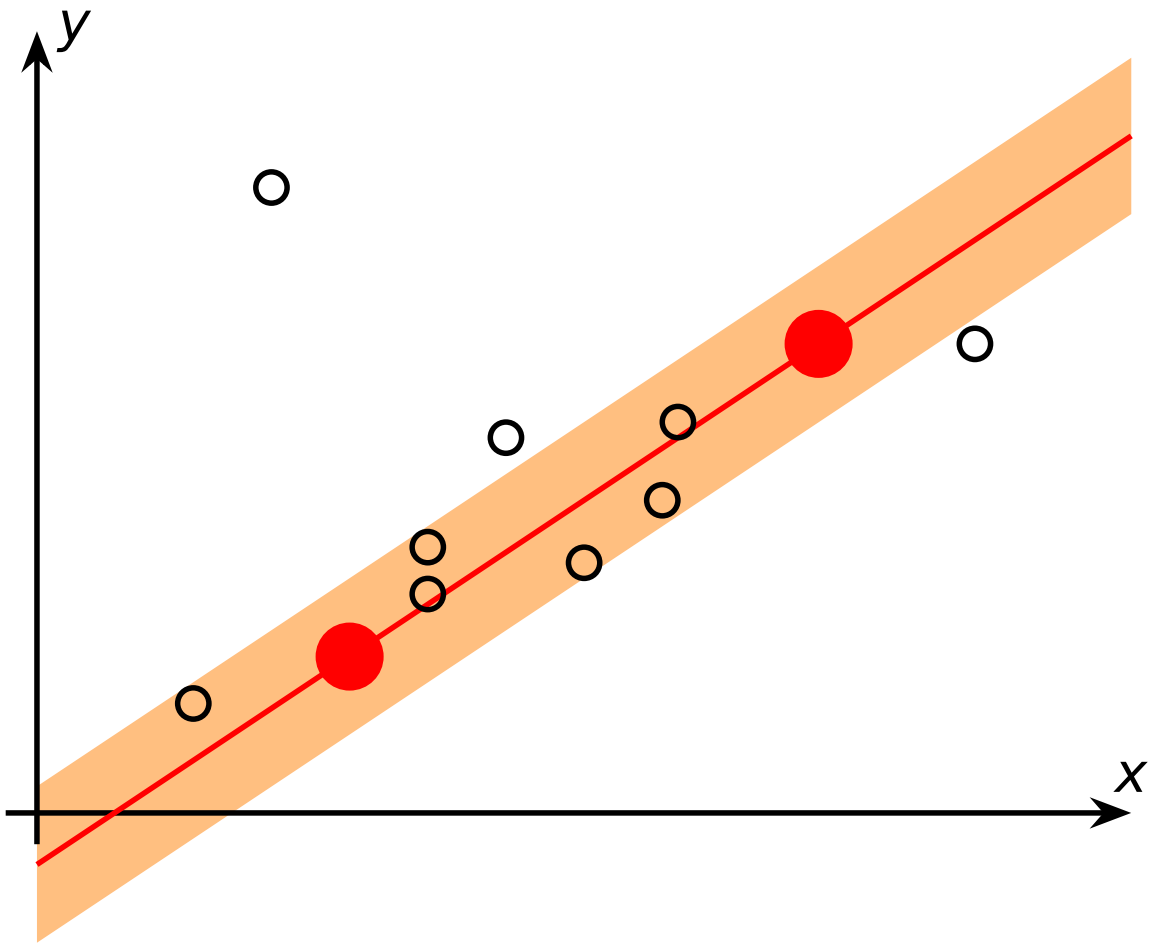
Выберем другую случайную пару

Пример работы RANSAC



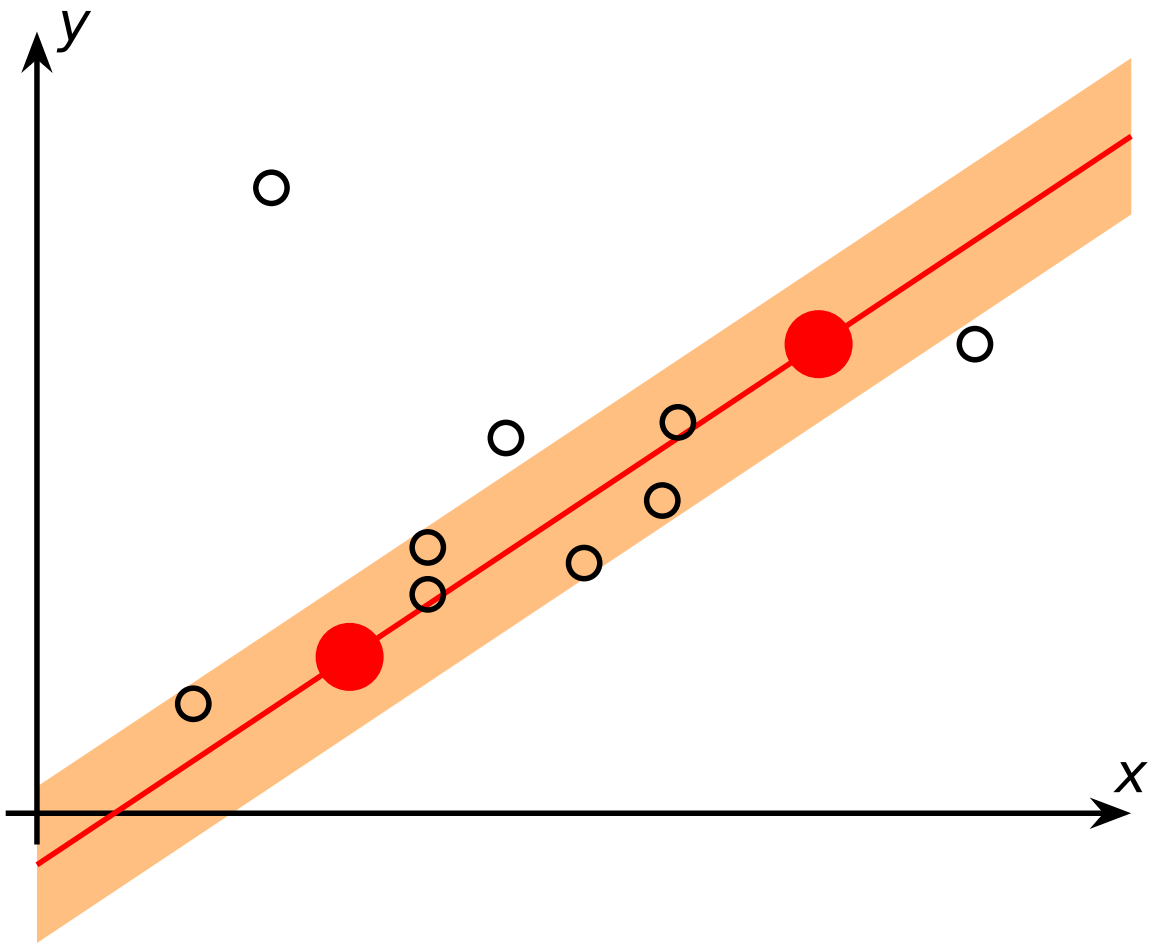
Выберем другую случайную пару

Пример работы RANSAC



Опять проведём между выбранными точками полосу шириной ρ и посмотрим, сколько точек попало в неё

Пример работы RANSAC



Видно, что таких точек в данном случае больше. Если их доля выше ψ, линия найдена, иначе пытаемся дальше

Свойства RANSAC

Вопрос: обязательно ли найдётся решение?

Свойства RANSAC

Вопрос: обязательно ли найдётся решение?
Ответ: нет. Может повезти сразу, может — через много попыток, может — никогда.

Вопрос: можно ли так же подбирать другие кривые, например окружности?

Свойства RANSAC

Вопрос: обязательно ли найдётся решение?
Ответ: нет. Может повезти сразу, может — через много попыток, может — никогда.

Вопрос: можно ли так же подбирать другие кривые, например окружности?
Ответ: можно; надо только брать нужное количество точек для однозначного задания кривой.

Достоинства: простой, прямолинейный, универсальный подход; на практике часто работает на удивление хорошо; устойчив к выбросам.

Недостатки: наличие параметров ρ и ψ, которые надо как-то подбирать; случайный характер успеха.

Сплайновая интерполяция

Можно аппроксимировать функцию сплайном — набором полиномов малой степени, гладко переходящих друг в друга. При этом задача формально сводится к линейной регрессии параметров этих сплайнов.

Для простоты рассмотрим только одномерный случай, т.е. аппроксимацию функции $\mathbb{R} \to \mathbb{R}$. Как пример рассмотрим кубические полиномы:

$$y = c_0^i + c_1^i x + c_2^i x^2 + c_3^i x^3$$

Пусть все известные прецеденты расположены на отрезке $[x_{start}, x_{finish}]$. Они делят отрезок на части, в каждой из которых будет создан свой полином.

Сплайновая интерполяция (2)

Запишем для каждой точки равенства:

- значений двух соседних полиномов;
- их первых производных;
- их вторых производных.

Получим для каждой точки три уравнения, в которые коэффициенты c входят как в линейные. Получим систему уравнений, из которой найдём коэффициенты сплайнов.

Пример вычисления сплайнов

Построим сплайн через те же точки, для которых считали линейную регрессию: (2,1), (4,2), (5,3).

Вопрос: сколько в него будет входить отдельных кубических полиномов?

Пример вычисления сплайнов

Построим сплайн через те же точки, для которых считали линейную регрессию: (2,1), (4,2), (5,3).

Вопрос: сколько в него будет входить отдельных кубических полиномов?
Ответ: два: один для отрезка [2,4] и один для отрезка [4,5].

Пусть они имеют вид:

$$f_1(x) = c_0^1 + c_1^1 x + c_2^1 x^2 + c_3^1 x^3, 2 \leq x \leq 4$$

$$f_2(x) = c_0^2 + c_1^2 x + c_2^2 x^2 + c_3^2 x^3, 4 \leq x \leq 5$$

Вычислим коэффициенты

Расчёт коэффициентов

Функции должны пройти через точки, т.е. $f_1(2) = 1$, $f_1(4) = f_2(4) = 2$, $f_2(5) = 3$:

$$
\begin{aligned}
c_0^1 + 2c_1^1 + 4c_2^1 + 8c_3^1 &= 1 \\
c_0^1 + 4c_1^1 + 16c_2^1 + 64c_3^1 &= 2 \\
c_0^2 + 4c_1^2 + 16c_2^2 + 64c_3^2 &= 2 \\
c_0^2 + 5c_1^2 + 25c_2^2 + 125c_3^2 &= 3
\end{aligned}
$$

Первая и вторая производные в центральной точке должны быть равны. Первые производные на краях приравняем к нулям:

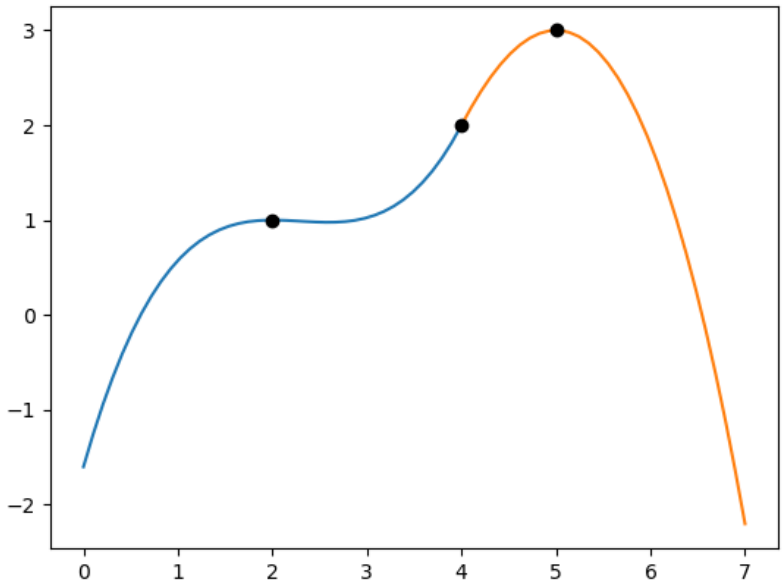
$$
\begin{aligned}
f_1'(4) = f_2'(4) &= c_1^1 + 8c_2^1 + 48c_3^1 = c_1^2 + 8c_2^2 + 48c_3^2 \\
f_1''(4) = f_2''(4) &= 2c_2^1 + 12c_3^1 = 2c_2^1 + 12c_3^1 \\
f_1'(2) &= c_1^1 + 4c_2^1 + 12c_3^1 = 0 \\
f_2'(5) &= c_1^2 + 10c_2^2 + 75c_3^2 = 0
\end{aligned}
$$

Расчёт коэффициентов (2)

Получаем систему из 8 линейных уравнений с 8 неизвестными:

$$\begin{pmatrix} 1 & 2 & 4 & 8 & 0 & 0 & 0 & 0 \\ 1 & 4 & 16 & 64 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 16 & 64 \\ 0 & 0 & 0 & 0 & 1 & 5 & 25 & 125 \\ 0 & 1 & 8 & 48 & 0 & -1 & -8 & -48 \\ 0 & 0 & 2 & 12 & 0 & 0 & -2 & -12 \\ 0 & 1 & 4 & 12 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 10 & 75 \end{pmatrix} \begin{pmatrix} c_0^1 \\ c_1^1 \\ c_2^1 \\ c_3^1 \\ c_0^2 \\ c_1^2 \\ c_2^2 \\ c_3^2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 2 \\ 3 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Итоговый сплайн



Формулы частей сплайна:

$$f_1(x) = -1.6 + 3.5x - 1.55x^2 + 0.225x^3$$
$$f_2(x) = -12 + 3.5x + 0.4x^2 - 0.1x^3$$

Видно, что сплайн точно проходит через все точки и что он гладкий.

Уравнения в общем виде

Пусть сплайн должен проходить через N точек: (x_1, y_1), (x_2, y_2) ... (x_N, y_N), причём $x_1 < x_2 < ... < x_N$. Тогда кубический сплайн будет состоять из $N - 1$ кусков:

$$f_1(x) = c_0^1 + c_1^1 x + c_2^1 x^2 + c_3^1 x^3, x_1 \leq x \leq x_2$$

$$f_2(x) = c_0^2 + c_1^2 x + c_2^2 x^2 + c_3^2 x^3, x_2 \leq x \leq x_3$$

...

$$f_{N-1}(x) = c_0^{N-1} + c_1^{N-1} x + c_2^{N-1} x^2 + c_3^{N-1} x^3, x_{N-1} \leq x \leq x_N$$

Требуются равенства значений и производных:

$$\forall i \in 1..N-1 : f_i(x_{i+1}) = y_{i+1} = f_{i+1}(x_{i+1})$$

$$\forall i \in 2..N-1 : f_i'(x_{i+1}) = f_{i+1}'(x_{i+1}), f_i''(x_{i+1}) = f_{i+1}''(x_{i+1})$$

Пример уравнений сплайна(2)

$$c_0^1 + c_1^1 x_1 + c_2^1 x_1^2 + c_3^1 x_1^3 = y_1$$

$$c_0^1 + c_1^1 x_2 + c_2^1 x_2^2 + c_3^1 x_2^3 = y_2$$

$$c_0^2 + c_1^2 x_2 + c_2^2 x_2^2 + c_3^2 x_2^3 = y_2$$

$$c_0^2 + c_1^2 x_3 + c_2^2 x_3^2 + c_3^2 x_3^3 = y_3$$

...

$$c_1^1 + 2c_2^1 x_2 + 3c_3^1 x_2^2 = c_1^2 + 2c_2^2 x_2 + 3c_3^2 x_2^2$$

...

$$2c_2^1 + 6c_3^1 x_2 = 2c_2^2 + 6c_3^2 x_2$$

Для полного числа неизвестных не хватает ещё двух уравнений. Можно, например, наложить дополнительные условия на производные в краевых точках (равенство нулю или между собой).

Выводы по сплайнам

Достоинства: точные прохождения через примеры, гладкость, достаточно быстро считаются, расширяемы на множество измерений, существуют всегда.

Недостатки: никто не гарантирует хорошего поведения вне точек (очень часто при большом числе точек происходят резкие скачки), полная неустойчивость к выбросам, коэффициентов 1) много и 2) они плохо интерпретируются.

Ядерное сглаживание: идея метода

Если не нужно получать в явном виде уравнения аппроксимирующей функции (как у сплайнов), можно попытаться аппроксимировать зависимость из следующих соображений:

- чем ближе точки выборки к месту, для которого рассчитывается функция, тем сильнее они должны влиять на её значение;
- влияние далёких точек, скорее всего, будет очень слабым.

Попытаемся получить аппроксимирующую функцию в заданной точке x как некоторую линейную комбинацию всех N значений выборки, взвешенных по признаку близости к x:

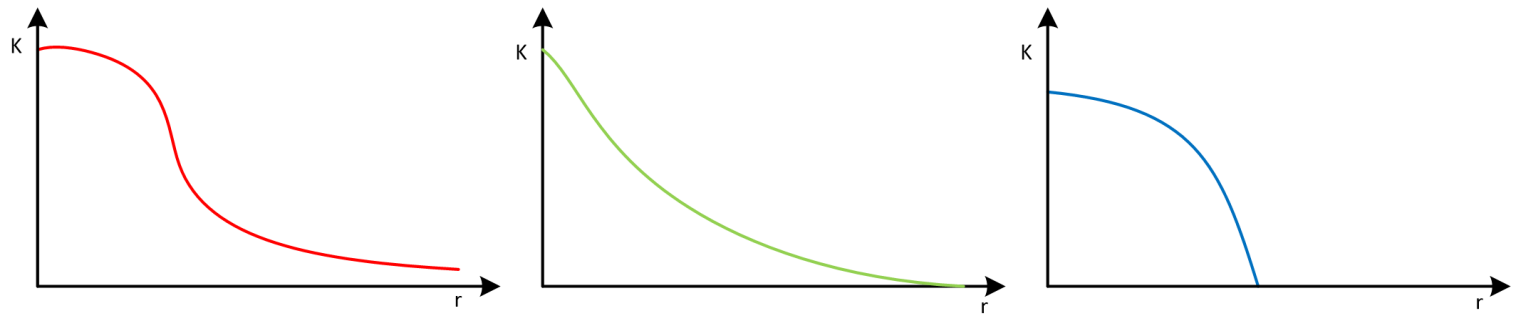
$$g(x) = \sum_{i=1}^{N} K(x_i - x) f(x_i)$$

Введём дополнительную функцию, определяющую зависимость «веса» точки от расстояния до неё — ядро K.

Ядро

Ядром может быть любая функция, которая:

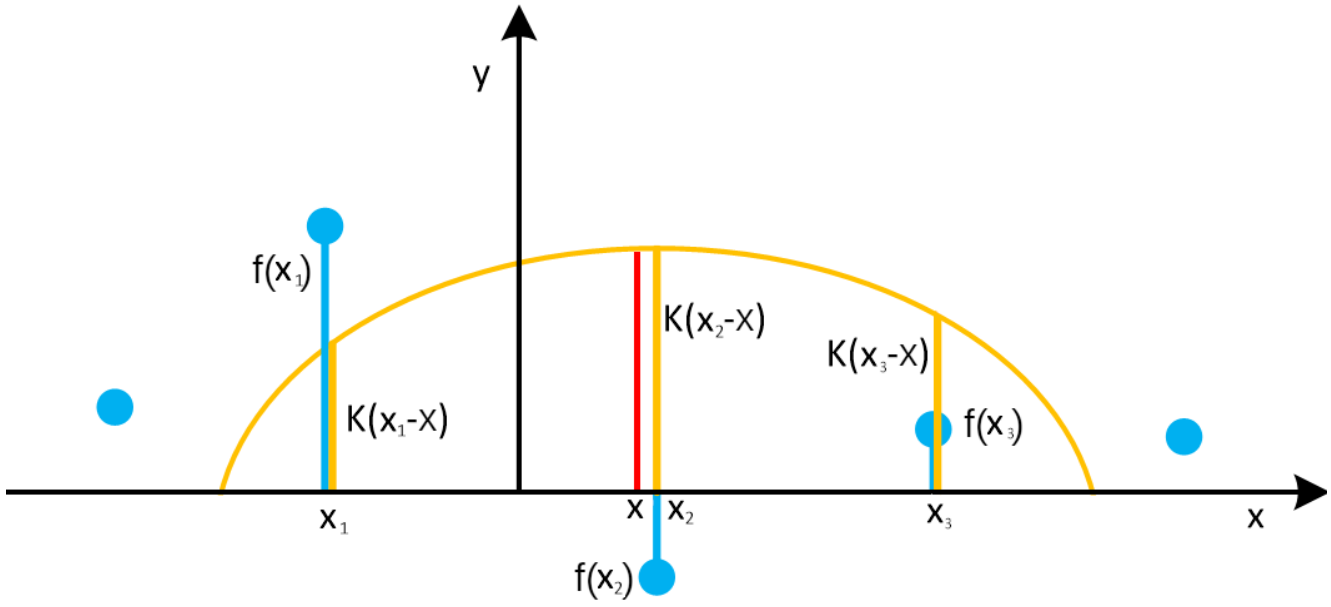
- определяет зависимость веса от расстояния: $[0, \text{inf}) \xrightarrow{K} [0, \text{inf}]$;
- является гладкой (чтобы результирующая функция тоже была гладкой);
- ограничена;
- не возрастает.



Часто используется, например, гауссиана.

Расчёт результата

Ядро выбирается один раз и не меняется в процессе работы:



$$g(x) = \frac{\ldots + K(x_1 - x)f(x_1) + K(x_2 - x)f(x_2) + K(x_3 - x)f(x_3) + \ldots}{\ldots + K(x_1 - x) + K(x_2 - x) + K(x_3 - x) + \ldots}$$

Влияние параметров

Вопрос: как влияет на результат ширина ядра?

Влияние параметров

Вопрос: как влияет на результат ширина ядра?
Ответ: при очень крутом (узком) ядре функция сильно следует каждой конкретной точке (переобучение); при очень пологом — слабо следует точкам, чрезмерно их усредняя и сглаживая (недообучение).

Вопрос: сильно ли влияет на результат конкретная форма ядра?

Влияние параметров

Вопрос: как влияет на результат ширина ядра?
Ответ: при очень крутом (узком) ядре функция сильно следует каждой конкретной точке (переобучение); при очень пологом — слабо следует точкам, чрезмерно их усредняя и сглаживая (недообучение).

Вопрос: сильно ли влияет на результат конкретная форма ядра?
Ответ: как ни странно, нет — только на гладкость.

Почему формула ядерного сглаживания именно такая?

Для немного более строгого обоснования указанной формулы ядерного сглаживания попытаемся минимизировать сумму квадратов его ошибок, но в модифицированном виде — «взвесив» каждое слагаемое.

Пусть в выборке N точек $(x_i, y_i), i = 1..N$; при этом в каждой из них значение сглаженной функции равно $g(x_i)$. Сумма квадратов ошибок для всех точек равна

$$L = \sum_{i=1}^{N} (g(x_i) - y_i)^2$$

Припишем к каждому слагаемому некоторый вес ω_i, определяющий его значимость для минимизируемой ошибки:

$$L = \sum_{i=1}^{N} \omega_i (g(x_i) - y_i)^2$$

Почему формула ядерного сглаживания именно такая? (2)

$$L = \sum_{i=1}^{N} \omega_i (g(x_i) - y_i)^2$$

Если минимизировать эту L сразу для всех точек вместе, получится общая задача регрессии. Вместо этого попытаемся минимизировать ошибку только для одной точки x, т.е. найти оптимальный вариант строго для неё, без привязки к остальным точкам. Для этого примем $\omega_i = K(x - x_i)$ (т.е. значимость точки x_i для ошибки падает с расстоянием до x) и получим новую функцию ошибки:

$$\mathbb{L} = \sum_{i=1}^{N} K(x - x_i)(g(x_i) - y_i)^2$$

Почему формула ядерного сглаживания именно такая? (3)

$$\mathbb{L} = \sum_{i=1}^{N} K(x - x_i)(g(x_i) - y_i)^2$$

В точках экстремума частные производные этой функции по её параметрам равны нулю. Сделаем ещё одно (очень сильное) предположение: пусть функция $g(x_i)$ не слишком отличается для разных x, т.е. представляет собой числовую константу $g(x) \forall x$. Возьмём производную по ней и приравняем к нулю, чтобы найти экстремум $g(x_i)$:

$$\frac{\partial \mathbb{L}}{\partial g(x)} = 2 \sum_{i=1}^{N} K(x - x_i)(g(x) - y_i) = 0$$

$$\sum_{i=1}^{N} K(x - x_i) g(x) = \sum_{i=1}^{N} K(x - x_i) y_i$$

Формула Надарая-Уотсона

Выносим g(x) за знак суммы и получаем:

$$g(x)\sum_{i=1}^{N}K(x-x_i)=\sum_{i=1}^{N}K(x-x_i)y_i$$

и

$$g(x)=\frac{\sum\limits_{j=1}^{N}K(x-x_j)y_j}{\sum\limits_{j=1}^{N}K(x-x_j)}$$

Такой вид $g(x)$ называется формулой Надарая-Уотсона. Для более строгого вывода (без таких сильных предположений) можно использовать теорию вероятности.

Есть и другие формулы для сглаживания (Пристли-Чжао, Гассера-Мюллера); их результаты во многих случаях близки.

Подбор гиперпараметра

Ширина ядра h — параметр, который не меняется в процессе обучения и работы алгоритма. Такие параметры называются гиперпараметрами. Их подбор — важная часть решения задач машинного обучения.

Примеры подбора:

- ручной скользящий контроль: вносить плавные изменения в гиперпараметр, добиваясь улучшения результатов путём их визуального сравнения;
- GridSearch: поиск наилучшего решения задачи путём перебора всех возможных сочетаний значений гиперпараметров;
- HalvingGridSearch: то же, но после подбора наилучшего набора продолжает поиск в его окрестности с меньшим шагом;
- RandomizedSearch: пробы случайных значений с отбором наилучших итоговых результатов.

Подробнее — см. «Hyper-parameter optimizers» или средства модуля sklearn.model_selection.

Вывод по ядерному сглаживанию

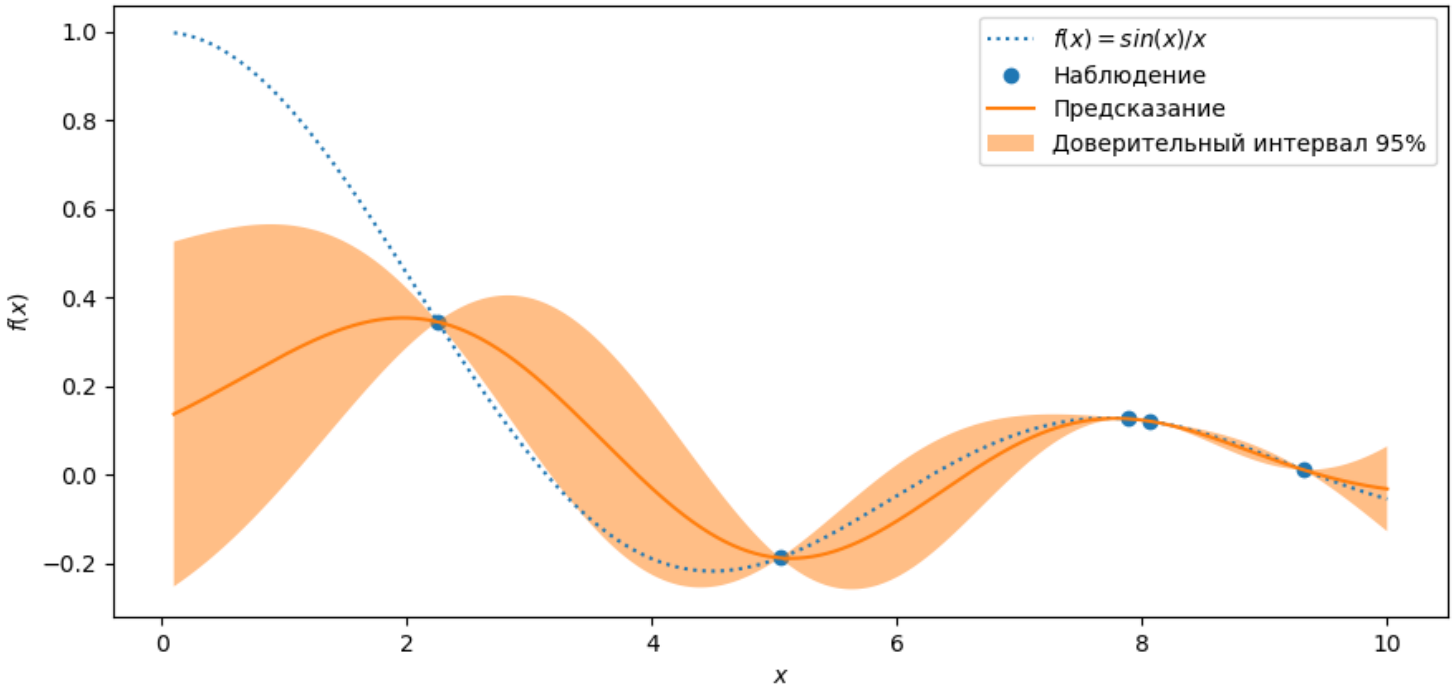
Достоинства: сработает всегда, результат достаточно гладкий

Недостатки: форма ядра берётся наугад, ширина — подгоняется. Для вычисления одного значения в общем случае нужна вся обучающая выборка, короткой формы записи результата нет.

Ещё один вариант регрессии — гауссовский процесс

До этого мы отвлекались от физического смысла приближаемой функции. Попробуем считать, что она — зависимость от времени какой-то случайной величины.

Раз в некоторых точках мы знаем её значения, то можно оценить их и в окрестностях этих точек.



Опишем зависимость как возникшую случайно

Случайный процесс $\xi(w, t)$ — множество случайных величин, каждой из которых приписано значение $t \in \mathbb{R}$ — время получения. w — парамеры процесса.

Если зафиксировать время t, получим одну случайную величину $\xi_t(w)$, зависящую от параметров w; если зафиксировать w, получится функция (обычная, без случайных факторов) только от времени $\xi_w(t)$.

Смысл введения понятия — описать совместное действие случайных факторов и функциональной зависимости.

Параметры случайного процесса

Среднее значение — вычисляется усреднением по всем возможным w, т.е. зависит только от времени: $\mu(t) = \mathbb{E}[\xi(w, t)]$

Ковариационная функция — ковариация между значениями процесса в два момента, рассчитанная по всем возможных w: $C(t_1, t_2) = Cov(\xi(w, t_1), \xi(w, t_2))$; можно ввести и дисперсию: $Var[t] = C(t, t)$

Вопрос: если $C(t_1, t_2)$ при близких t_1 и t_2 близка к нулю, как ведёт себя процесс?

Параметры случайного процесса

Среднее значение — вычисляется усреднением по всем возможным w, т.е. зависит только от времени: $\mu(t) = \mathbb{E}[\xi(w, t)]$

Ковариационная функция — ковариация между значениями процесса в два момента, рассчитанная по всем возможных w: $C(t_1, t_2) = Cov(\xi(w, t_1), \xi(w, t_2))$; можно ввести и дисперсию: $Var[t] = C(t, t)$

Вопрос: если $C(t_1, t_2)$ при близких t_1 и t_2 близка к нулю, как ведёт себя процесс?
Ответ: это значит, что значения в разные моменты времени не связаны между собой, т.е. процесс беспорядочно скачет. Если для близких моментов ковариация растёт, он становится более плавным.

Гауссовский процесс

Если любая конечная комбинация значений процесса нормально распределена (т.е. её вероятность имеет гауссову плотность распределения), процесс называется гауссовым.

$$P(\xi(w,t_1),\xi(w,t_2),...,\xi(w,t_K)) = \mathcal{N}(\xi,\mu,\sigma)$$

Это значит, что вместо абстрактного набора параметров w можно рассматривать более конкретные значения μ и σ.

Рассмотрим вариант, когда среднее значение такого процесса не меняется, а ковариационная функция зависит только от разности двух моментов, для которых она вычисляется: $\mu = const, \forall \delta t : C(t, t+\delta t) = C(0, \delta t)$

Задача регрессии

Пусть имеются значения некоторой конкретной реализации процесса в известные моменты времени. Найдём вероятность каждого значения этой же реализации в любые моменты времени:

$$P(\xi(t) = x | \xi(t_1) = x_1, \xi(t_2) = x_2, ..., \xi(t_N) = x_N) = ?$$

Так как процесс гауссов, то совместное распределение всех указанных вероятностей нормально:

$$P(x_1, x_2, ..., x_N) = \mathcal{N}(\xi, \Sigma)$$

Здесь ξ — вектор средних значений процесса в моменты t_1, t_2, ..., t_N, а Σ — их матрица ковариации: $\Sigma_{ij} = C(t_i, t_j)$.

Вопрос: как будет выглядеть аналогичное распределение, если кроме t_1, t_2, ..., t_N учесть в нём ещё и некоторый другой момент t?

Решение задачи регрессии

$$P(x_1, x_2, ..., x_N) = \mathcal{N}(\xi, \Sigma)$$

Ответ: очень похоже, но с модифицированными ξ и Σ:

$$P(x, x_1, x_2, ..., x_N) = \mathcal{N}(\hat{\xi}, \hat{\Sigma})$$

Вектор $\hat{\xi}$ будет включать первым элементом среднее значение в момент t, а остальные значения совпадут с ξ. Матрица $\hat{\Sigma}$ будет включать в первой строке и первом столбце ковариации с моментом t, а остальная часть совпадёт с прежней матрицей Σ:

$$\hat{\xi} = (\xi(t), \xi_1, \xi_2, ..., \xi_N), \hat{\Sigma} = \begin{pmatrix} C(t,t) & C(t,t_1) & C(t,t_2) & ... & C(t,t_N) \\ C(t_1,t) & C(t_1,t_1) & C(t_1,t_2) & ... & C(t_1,t_N) \\ C(t_2,t) & C(t_2,t_1) & C(t_2,t_2) & ... & C(t_2,t_N) \\ ... & ... & ... & ... & ... \\ C(t_N,t) & C(t_N,t_1) & C(t_N,t_2) & ... & C(t_N,t_N) \end{pmatrix}$$

Решение задачи регрессии (2)

Кроме того, по формуле условной вероятности:

$$P(x, x_1, x_2, ..., x_N) = P(x_1, x_2, ..., x_N)P(x|x_1, x_2, ..., x_N)$$

Тогда можно выразить нужную нам вероятность как частное двух нормальных распределений:

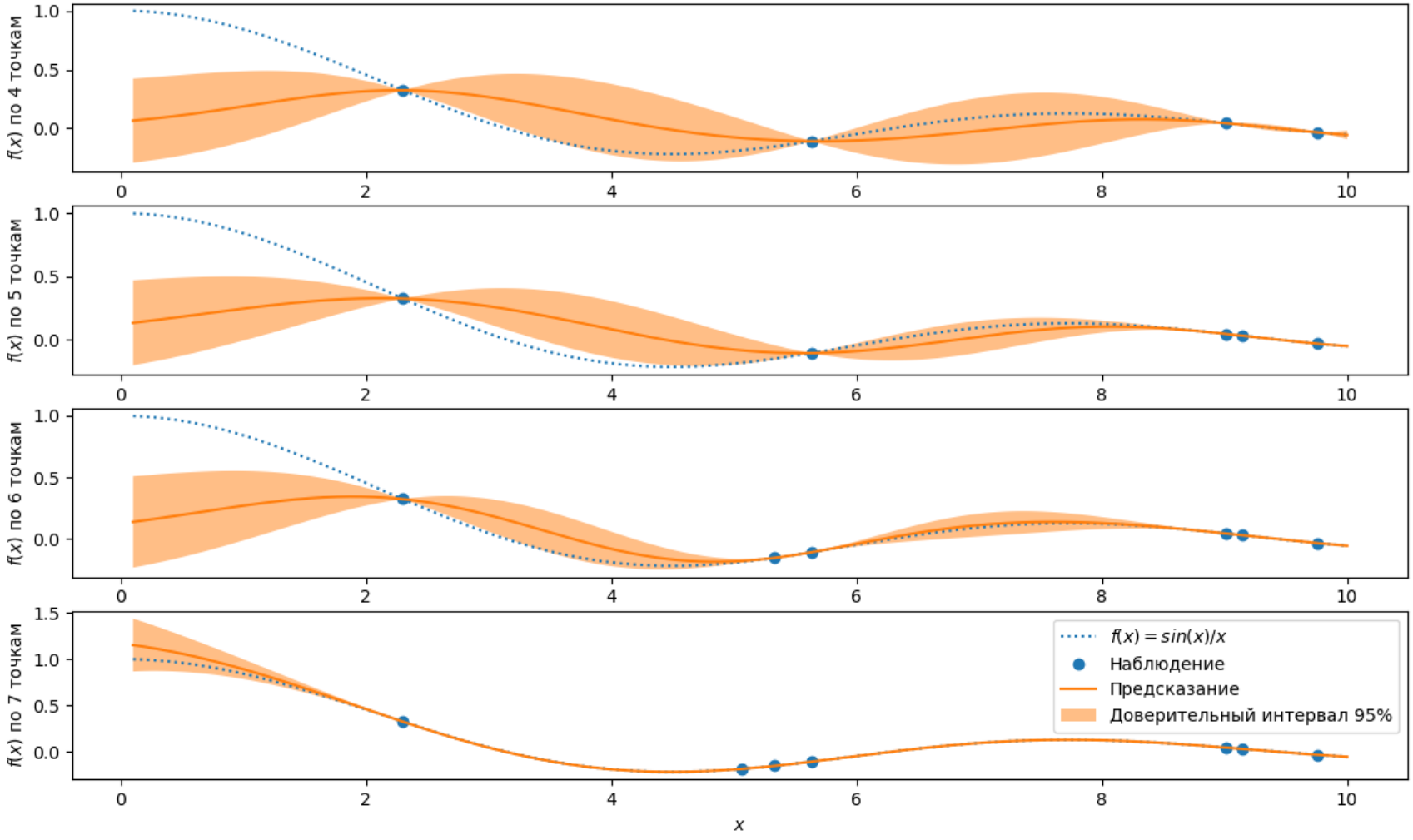
$$P(x|x_1, x_2, ..., x_N) = \frac{P(x, x_1, x_2, ..., x_N)}{P(x_1, x_2, ..., x_N)} = \frac{\mathcal{N}(\hat{\xi}, \hat{\Sigma})}{\mathcal{N}(\xi, \Sigma)}$$

Если использовать формулы для нормального распределения и обозначить $k = (C(t_1, t), C(t_2, t), ..., C(t_N, t))$, то можно рассчитать среднее значение и дисперсию для процесса в произвольный момент времени[2]:

$$\xi(t) = k\Sigma^{-1}t, Var(t) = C(t, t) - k\Sigma^{-1}k^T$$

[2] Детали см. в Wang J. An intuitive tutorial to Gaussian process regression

Примеры решений для разных входных данных



Достоинства и недостатки подхода

Достоинства: функции достаточно гладкие, параметров у них не очень много

Недостатки: предположения сильные, не всегда оправдываются.

Реализация моделей на scikit-learn

Работу 1 предлагается делать на Python с помощью библиотеки scikit-learn. Кроме неё, будут активно использоваться библиотеки Numpy и Matplotlib.

Работа со всеми моделями в scikit-learn делается по единому плану:

1. создание объекта, реализующего модель, с помощью его конструктора;
2. обучение модели методом fit;
3. получение результатов методом predict.

Пример линейной регрессии

```
# обучающая выборка
points = np.array([(0.3,3.3), (0.9,2.1), (1.2,2.0)]
lin_model = LinearRegression() # модель нужного типа
# обучение
res = lin_model.fit(points[:,0:1], points[:,1])
print(res.coef_) # коэффициенты регрессии
print(res.intercept_) # постоянный член
# получение результатов: входы
test_vals = np.linspace(0,2,50).reshape(-1,1)
# расчёт результатов на входах
test_res = lin_model.predict(test_vals)
# вывод графика
plt.plot(test_vals, test_res)
plt.show()  # показать график
```

Тема 3

Линейная классификация

Линейная классификация

Постановка задачи
Пример работы с показателями точности
Линейный классификатор
Метод опорных векторов
Задача квадратичной оптимизации в SVM
Пример классификации на SVM
Постановка задачи и её решение
Kernel trick

Задача классификации

Неформально: если для всякого элемента выборки известен его класс, создать метод для вычисления этого класса для любых объектов —в том числе тех, которых в выборке нет.
Формально: регрессия с категориальным правильным ответом.

Пусть имеются:

- множество X;
- множество допустимых классов Y;
- целевая функция $f : X \to Y$, значения f известны только на выборке $Z \subset X$ (т.е. на подмножестве X).

Требуется найти такую решающую функцию, или модель $g : X \to Y$, чтобы она хорошо предсказывала f на всём множестве X.

От того, что математически означает «хорошо предсказывала», зависит используемый аппарат.

Круг решаемых проблем

Вопрос: когда бывает нужно понять, каков закон разделения данных на группы?

Круг решаемых проблем

Вопрос: когда бывает нужно понять, каков закон разделения данных на группы?
Ответ: например:

- для обработки результатов измерений: «изделия получаются качественными, если сумма величин a и b меньше 5.4»;
- при распознавании образов: «ирис со значениями параметров $a < 2, b < 5$ принадлежит к такому-то виду».

Как и в случае регрессии, можно приспособить линейные способы решения задачи для некоторых нелинейных случаев.

Допущения

Пока для простоты будем считать, что классов только два.
Вопрос: а что, если это не так?

Допущения

Пока для простоты будем считать, что классов только два.
Вопрос: а что, если это не так?
Ответ: можно рекурсивно отделять классы друг от друга по очереди (каждый раз по одному или дихотомией);

Также считаем, что ни один класс не содержит подклассов, все объекты принадлежат к какому-либо из классов и только к одному.
Вопрос: а что, если это не так?

Допущения

Пока для простоты будем считать, что классов только два.
Вопрос: а что, если это не так?
Ответ: можно рекурсивно отделять классы друг от друга по очереди (каждый раз по одному или дихотомией);

Также считаем, что ни один класс не содержит подклассов, все объекты принадлежат к какому-либо из классов и только к одному.
Вопрос: а что, если это не так?
Ответ: можно дополнить систему классов так, чтобы в итоговой версии они не пересекались и охватывали всё множество объектов.

Сложности с оценкой качества

Вопрос: как оценить точность классификации?

Сложности с оценкой качества

Вопрос: как оценить точность классификации?
Ответ: единого общепринятого показателя нет, если отдельные характеристики разных аспектов.
Классификаторы обычно не идеальны, но ошибаться можно по-разному.

Пример: допустим, необходимо распознавать по анализу крови признаки очень опасной болезни (классифицировать результаты по наличию болезни).
Вопрос: как лучше ошибиться: не заметить имеющуюся болезнь или ошибочно выявить её там, где реально её нет?

Сложности с оценкой качества

Вопрос: как оценить точность классификации?
Ответ: единого общепринятого показателя нет, если отдельные характеристики разных аспектов.
Классификаторы обычно не идеальны, но ошибаться можно по-разному.

Пример: допустим, необходимо распознавать по анализу крови признаки очень опасной болезни (классифицировать результаты по наличию болезни).
Вопрос: как лучше ошибиться: не заметить имеющуюся болезнь или ошибочно выявить её там, где реально её нет?
Ответ: общепринято, что лучше перестраховаться (т.е. ошибочно выявить болезнь, провести дополнительное обследование и убедиться, что её на самом деле нет).

Виды ответов

Ответ на вопрос «Принадлежит ли объект к указанному классу?» может быть:

- истинным положительным, TP (true positive);
- истинным отрицательным, TN (true negative);
- ложным положительным: ошибка I рода, FP (false positive);
- ложным отрицательным: ошибка II рода, FN (false negative).

Пусть система должна отвечать, является ли объект буквой русского алфавита. Тогда ответы будут классифицироваться так:

Объект	A	Й	Q	W
Ответ системы	⊕	⊖	⊕	⊖
Правильный ответ	⊕	⊕	⊖	⊖
Класс ответа	TP	FN	FP	TN

Некоторые показатели точности

$$Precision = \frac{TP}{TP + FP}$$

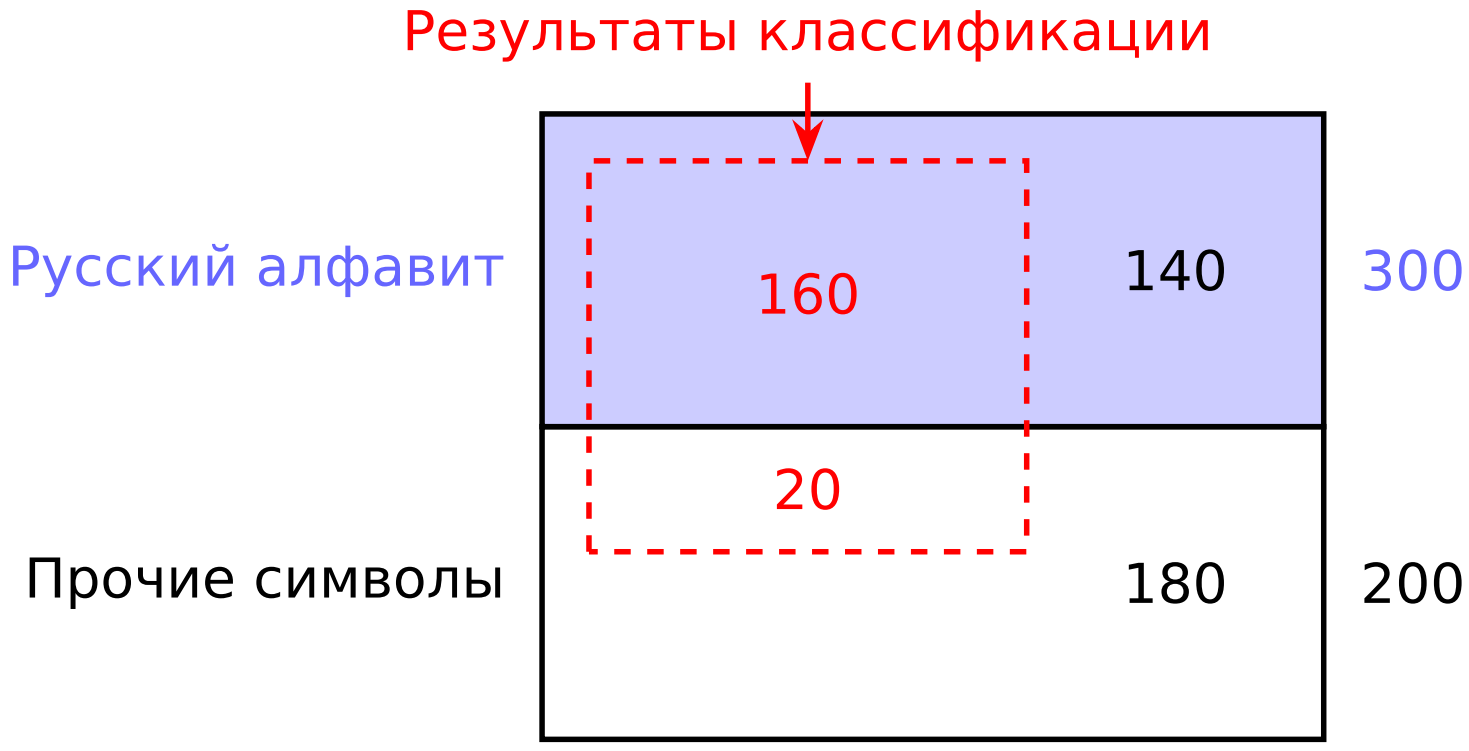
$$Recall = \frac{TP}{TP + FN}$$

$$Accuracy = \frac{TP + TN}{TP + FN + FP + FN}$$

С переводами их названий бывают проблемы: «Recall» обычно переводят как «полнота»; «precision» и «accuracy» обычно переводят как «точность», что вызывает путаницу, поэтому сейчас полезно использовать англоязычные термины.

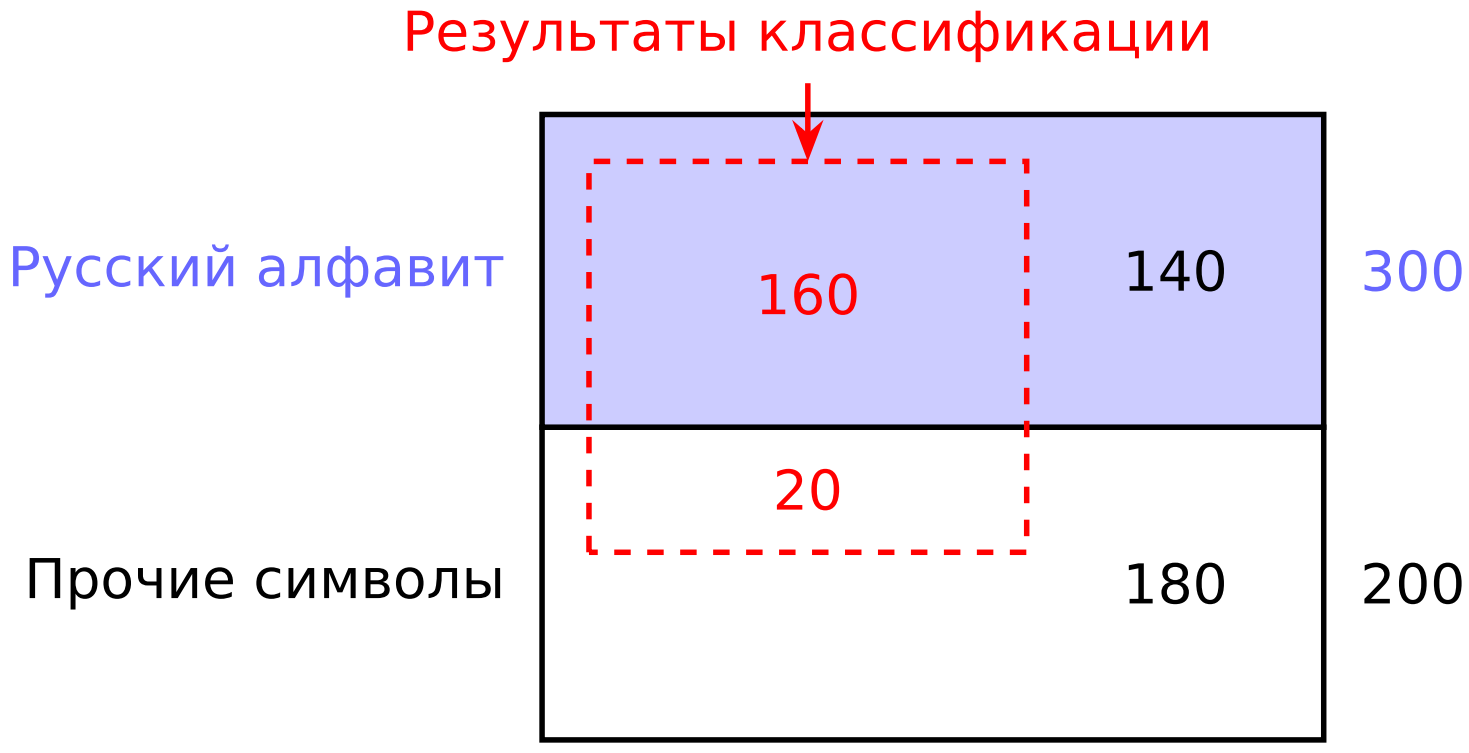
Accuracy вычисляется для всех классов сразу, precision и recall — для каждого класса отдельно от остальных.

Пример расчёта показателей точности



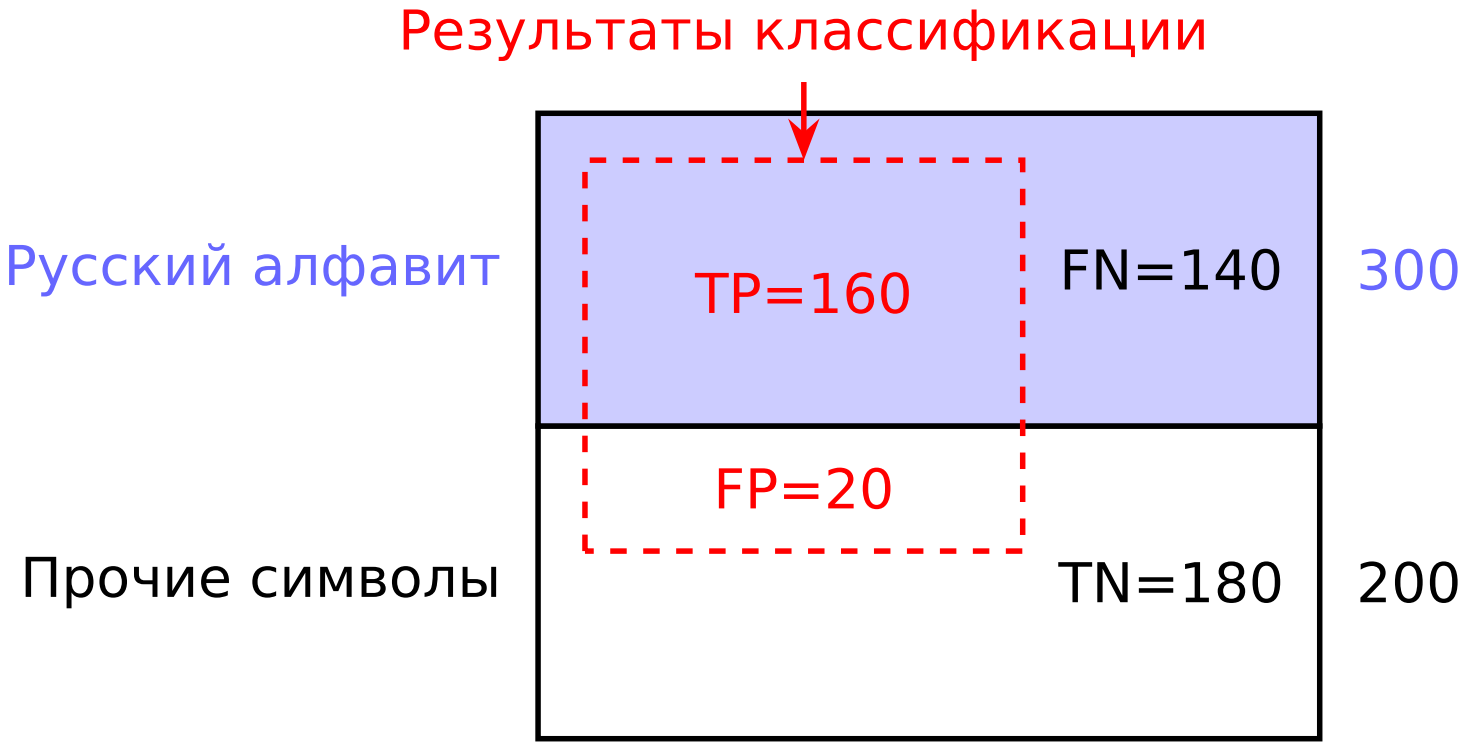
Пусть всего имеется 500 объектов, из них 300 относятся к некоторому классу. Классификатор назвал относящимися к этому классу 180 объектов — 160 правильно, 20 неправильно. Каковы показатели точности классификатора для данного класса?

Пример расчёта показателей точности (2)



TP=? FP=? TN=? FN=?

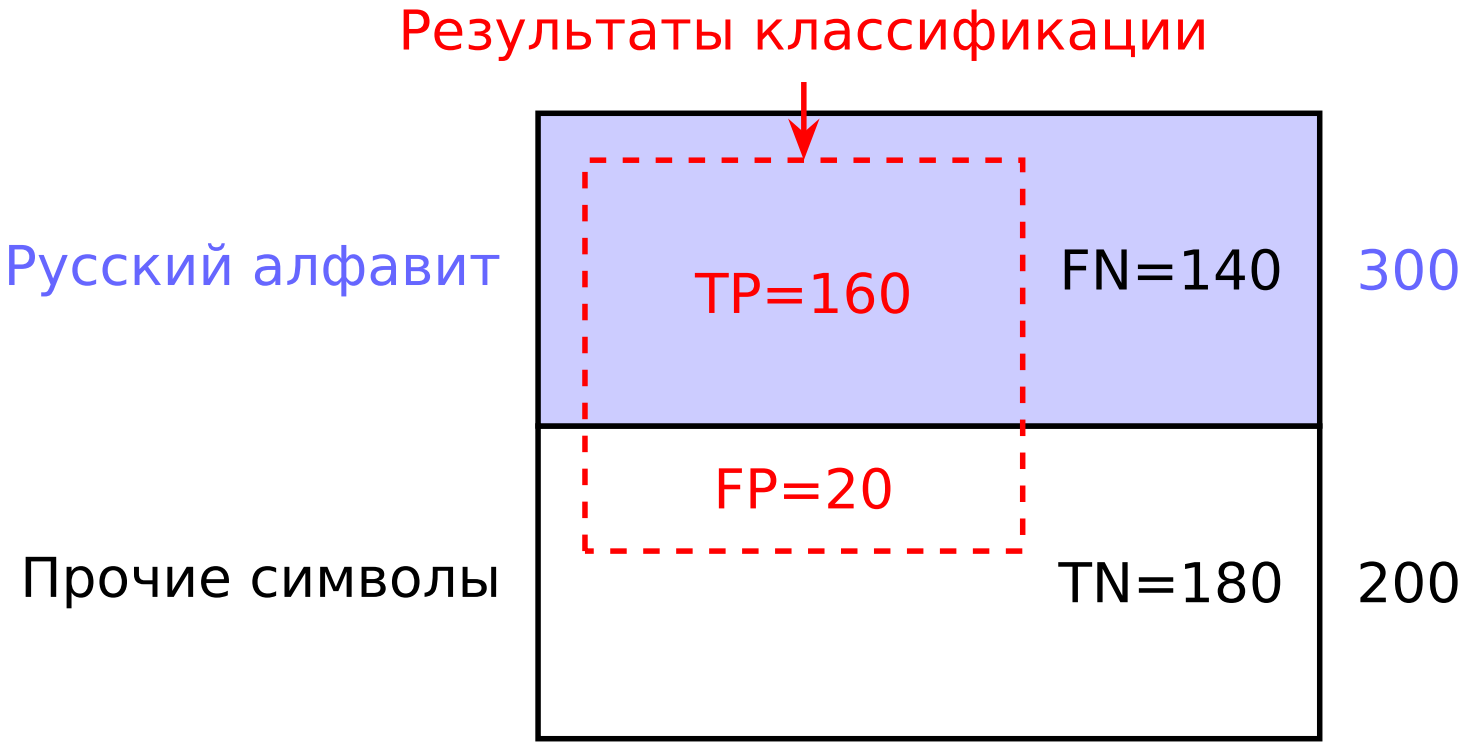
Пример расчёта показателей точности (2)



$$Precision = \frac{TP}{TP + FP} = ?; Recall = \frac{TP}{TP + FN} = ?$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = ?$$

Пример расчёта показателей точности (2)

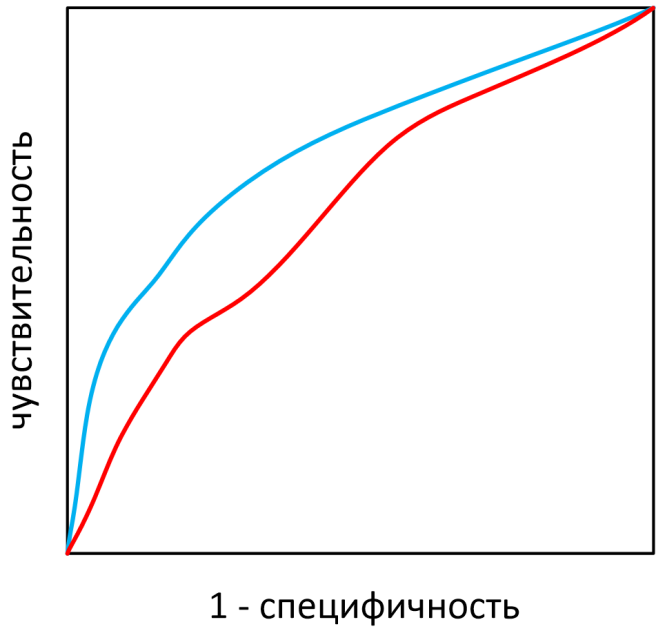


$$Precision = \frac{TP}{TP + FP} = \frac{160}{180}; Recall = \frac{TP}{TP + FN} = \frac{160}{300};$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{340}{500}$$

ROC-кривая

Пусть имеется метод с некоторым параметром «бдительности», задающим предпочтение ошибок первого или второго рода. Чтобы понять, как влияет на результат этот параметр, часто используется специальный график — ROC-кривая[3]

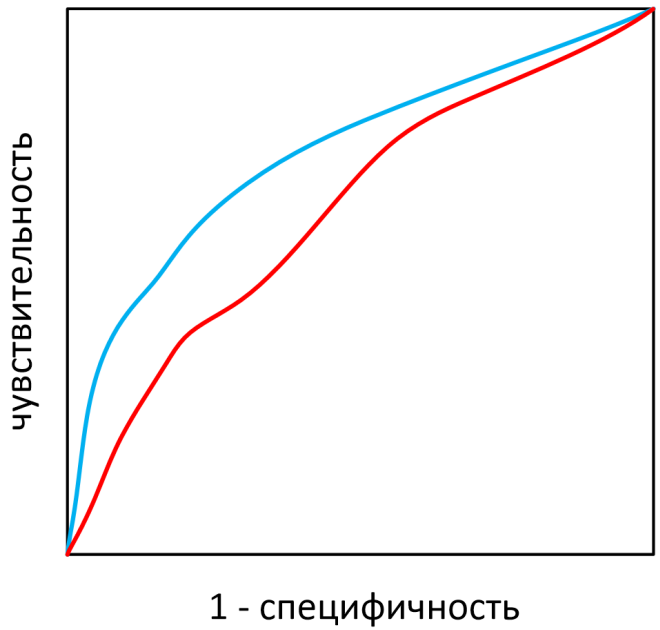


На этом графике показаны две такие кривые (для двух разных алгоритмов). По оси 0X — 1 - специфичность $\frac{FP}{FP+TN}$, по оси 0Y — полнота $\frac{TP}{TP+FN}$.

[3]от Receiver Operating Characteristic; название сохраняется по историческим причинам

ROC-кривая: физический смысл

Этот график характеризует качество алгоритма при изменение параметра.



Идея в том, что можно проследить изменения точности в зависимости от некоторого параметра. Обычно не удаётся просто снизить долю ложных предсказаний — можно либо «повысить бдительность» (чаще допускать ошибки I рода), либо «ослабить бдительность» (чаще допускать ошибки II рода).